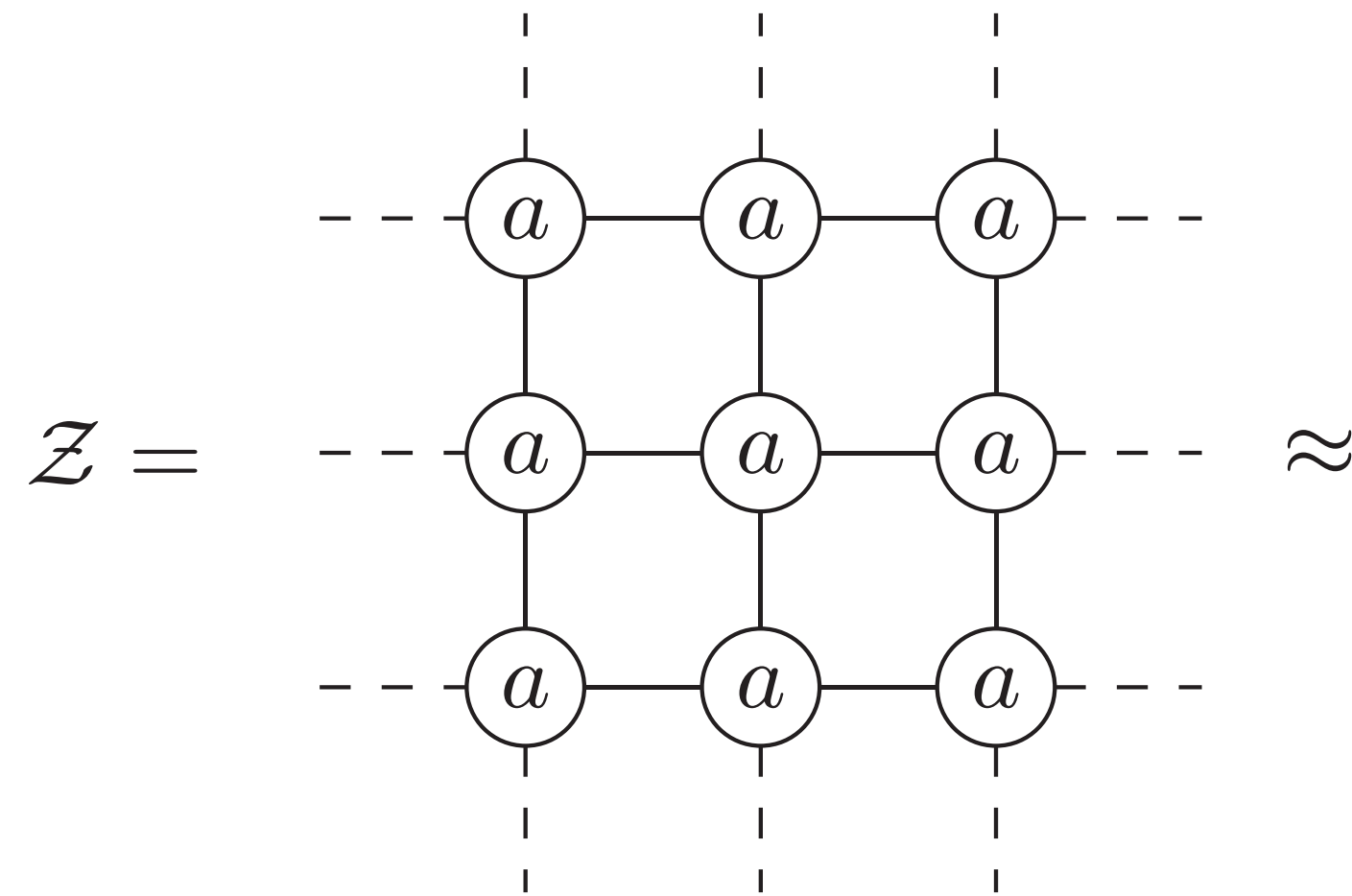


Parallel eigenvalue decomposition of real symmetric matrices

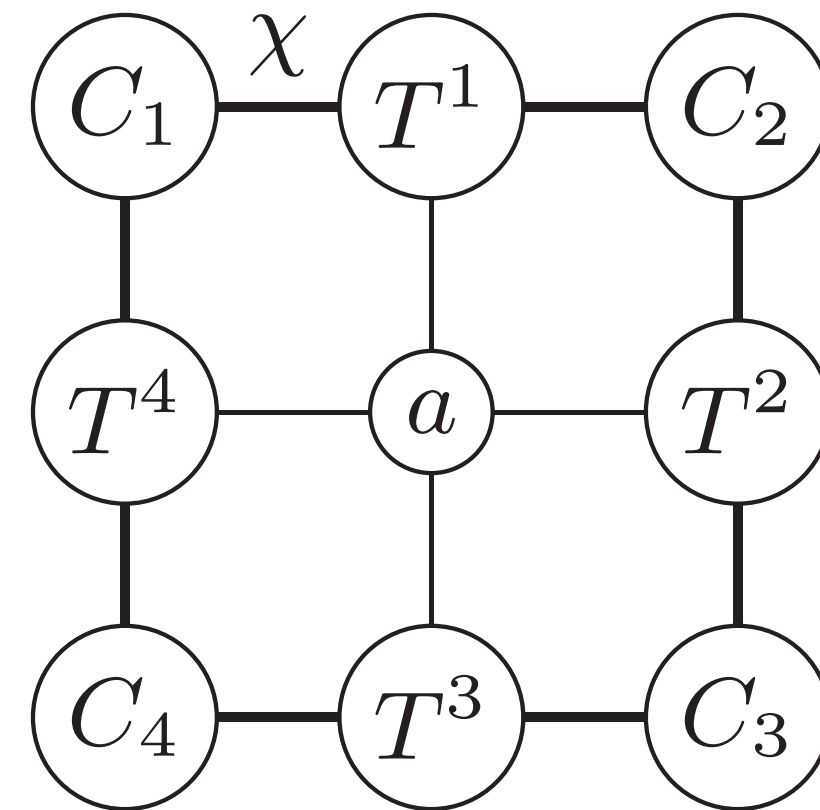
Afonso Rufino

EPFL

Physical motivation: CTMRG



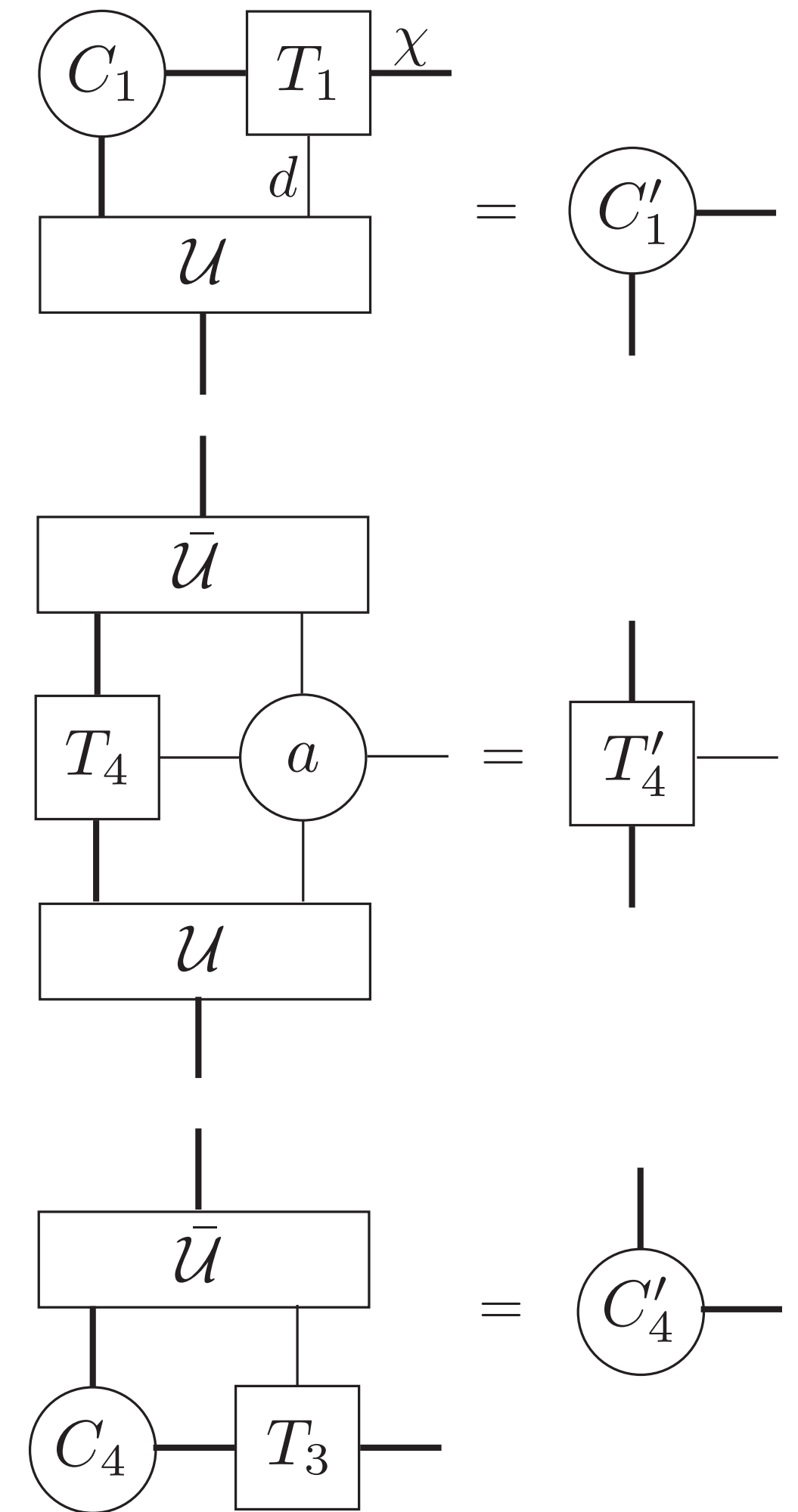
$\approx$



$$Z = \sum_{\{\sigma\}} \prod_{\langle i,j \rangle} e^{-\frac{J\sigma_i\sigma_j}{T}}$$

$$F = E - TS = -T \log Z$$

U determined through a singular value decomposition
(SVD) $M = USV^\dagger$. SVD can be calculated as the
eigenvalue decomposition of $A = M^\dagger M$



Nishino, T. & Okunishi, K. J. Phys. Soc. Jpn. 65, 891–894 (1996)

Nyckees, S., Rufino, A., Mila, F. & Colbois, J. Phys. Rev. E 108, 064132 (2023).

Eigenvalue decomposition of real symmetric matrices

Input: $A \in \mathbb{R}^{n \times n}, n \in \mathbb{N}, A = A^t$.

Output: $\{\Lambda, Z\}$ satisfying $A = Z^t \Lambda Z$, where Z is an orthogonal matrix ($ZZ^t = Z^t Z = I_{n \times n}$) and Λ is a diagonal matrix.

Numerical method

- **Householder method:** apply similarity transformations $A' = P(\mathbf{W})AP(\mathbf{W})$, with $P(\mathbf{W})_{i,j} = (\delta_{i,j} - 2W_i W_j)$, until A is tridiagonal.
- **QL iteration:** Determine $A = QL$, with Q an orthogonal matrix and L a lower triangular matrix, and apply the similarity transformation $A' = Q^t A Q$. As iterations are repeated, A converges to a diagonal matrix.

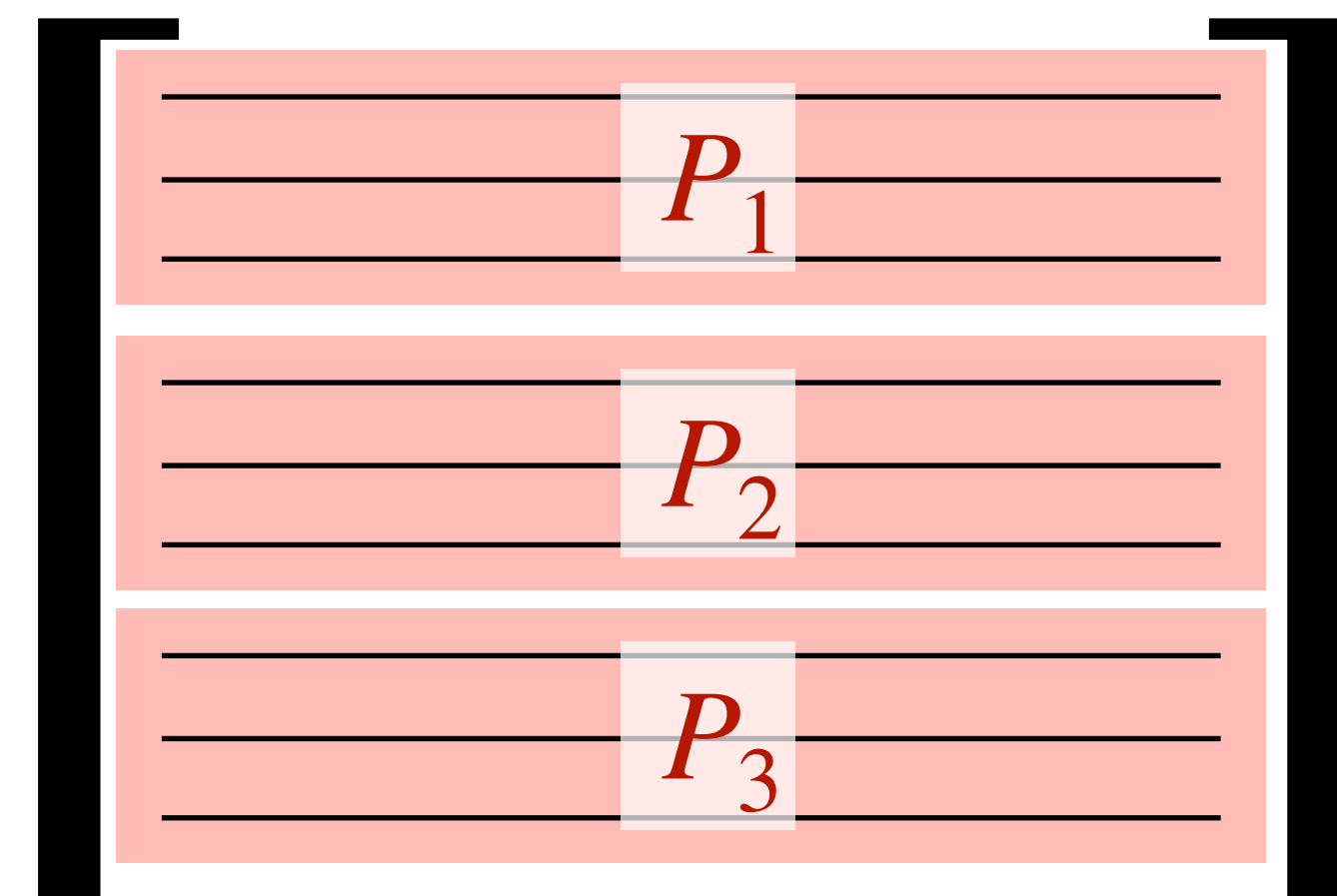
William H., Numerical Recipes in Fortran 77: The Art of Scientific Computing. (Cambridge University Press, Cambridge, 2007).

Wilkinson, J. H. & Reinsch, C. Handbook for Automatic Computation. (Springer Berlin Heidelberg, Berlin, Heidelberg, 1971)

Complexity and parallelization

Serial scaling

- **Householder:** $T \sim \alpha n^3$ (matrix multiplication).
- **QL:** $T \sim \gamma n^3 + \delta n^2$. If only eigenvalues are needed, $\gamma = 0$.



Parallel scaling

- For parallelisation, split the rows between the p processors. Householder can be fully parallelized, while in QL iteration only the eigenvector determination can.
- **Householder:** $T \sim \alpha n^3/p$. Must send information of size n between processors.
- **QL:** $T \sim \gamma n^3/p + \delta n^2$.

Amdahl's law

- Fraction of code which can be parallelized (assuming n large):

$$f = \frac{\alpha n^3 + \gamma n^3}{\alpha n^3 + \gamma n^3 + \delta n^2} = \frac{1}{1 + \frac{\delta}{(\alpha + \gamma)n}} \approx 1 - \frac{\delta}{(\alpha + \gamma)n} = 1 - \frac{\mu}{n}$$

$$S(p) = \frac{1}{1 - f + \frac{f}{p}} = \frac{p}{1 + \frac{p\mu}{n} \left(1 - \frac{1}{p}\right)} \rightarrow p$$

Implementation and profiling of serial code

- Code written in C.
- Randomly generate $n \times n$ real symmetric matrices (matrix elements sampled uniformly as $A_{i,j} = A_{j,i} \sim \mathcal{U}_{[-1,1]}$).
- Profiling with single thread and $n = 1600$.

Source Function Stack	CPU Time: Total ▼ »	CPU Time: Self »	Function (Full)	Source File
▼ Total	100.0%	0s		
▼ _start	100.0%	0s	_start	
▼ __libc_start_main	100.0%	0s	__libc_start_...	
▼ __libc_start_call_main	100.0%	0s	__libc_start_...	
▼ main	100.0%	0s	main	eigen_mai...
▶ QL_iteration	76.6%	0.010s	QL_iteration	eigen_seri...
▼ householder	23.3%	0.010s	householder	eigen_seri...
▶ formZ	10.3%	68.650s	formZ	eigen_seri...
▶ formV	8.0%	53.753s	formV	eigen_seri...
▶ formA	5.0%	33.440s	formA	eigen_seri...
▶ formW	0.0%	0.100s	formW	eigen_seri...
▶ formQ	0.0%	0.030s	formQ	eigen_seri...
▶ build_random_sym_matrix	0.0%	0s	build_rando...	eigen_seri...
▶ save_vec_hdf5	0.0%	0s	save_vec_hdf5	eigen_seri...

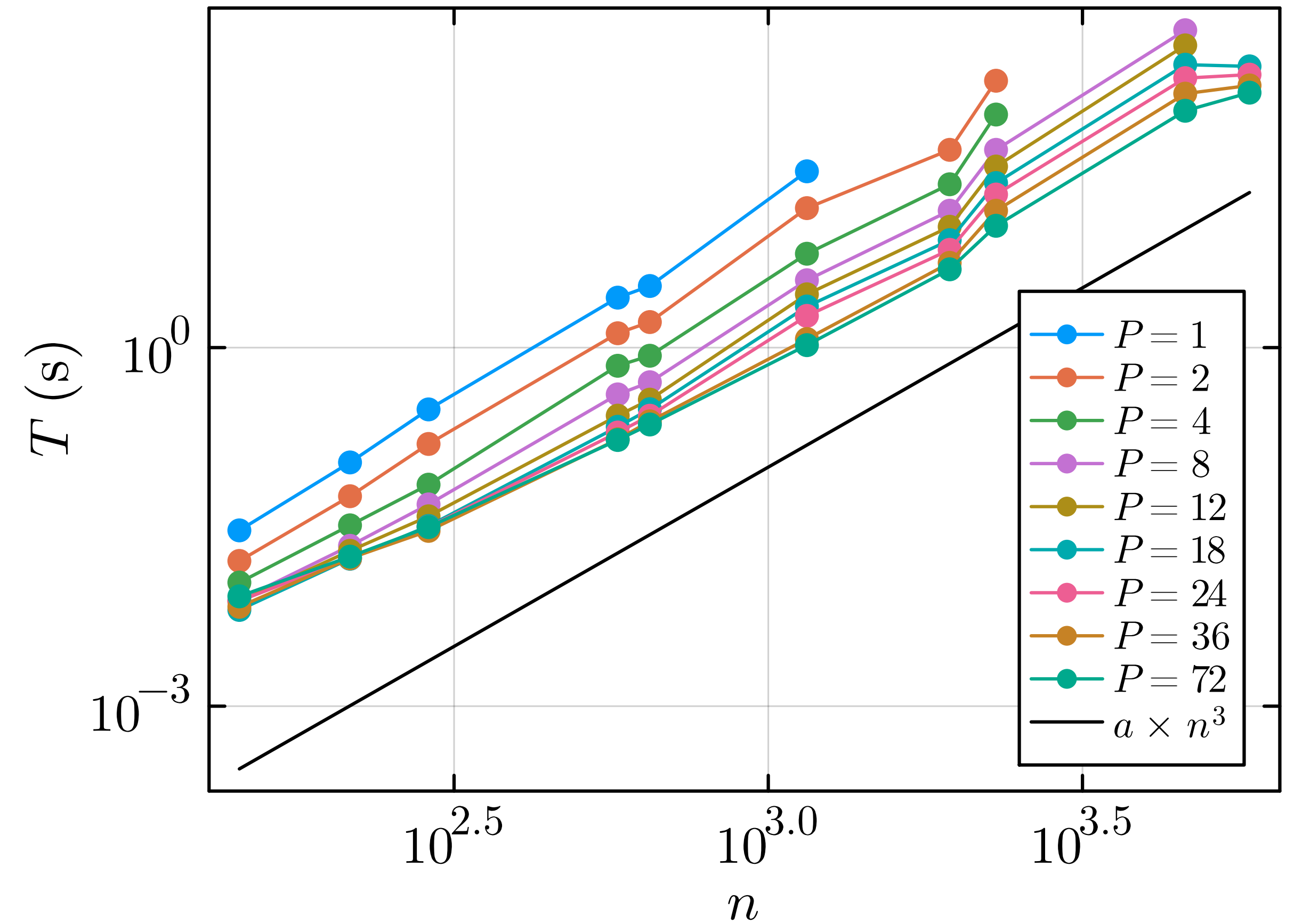
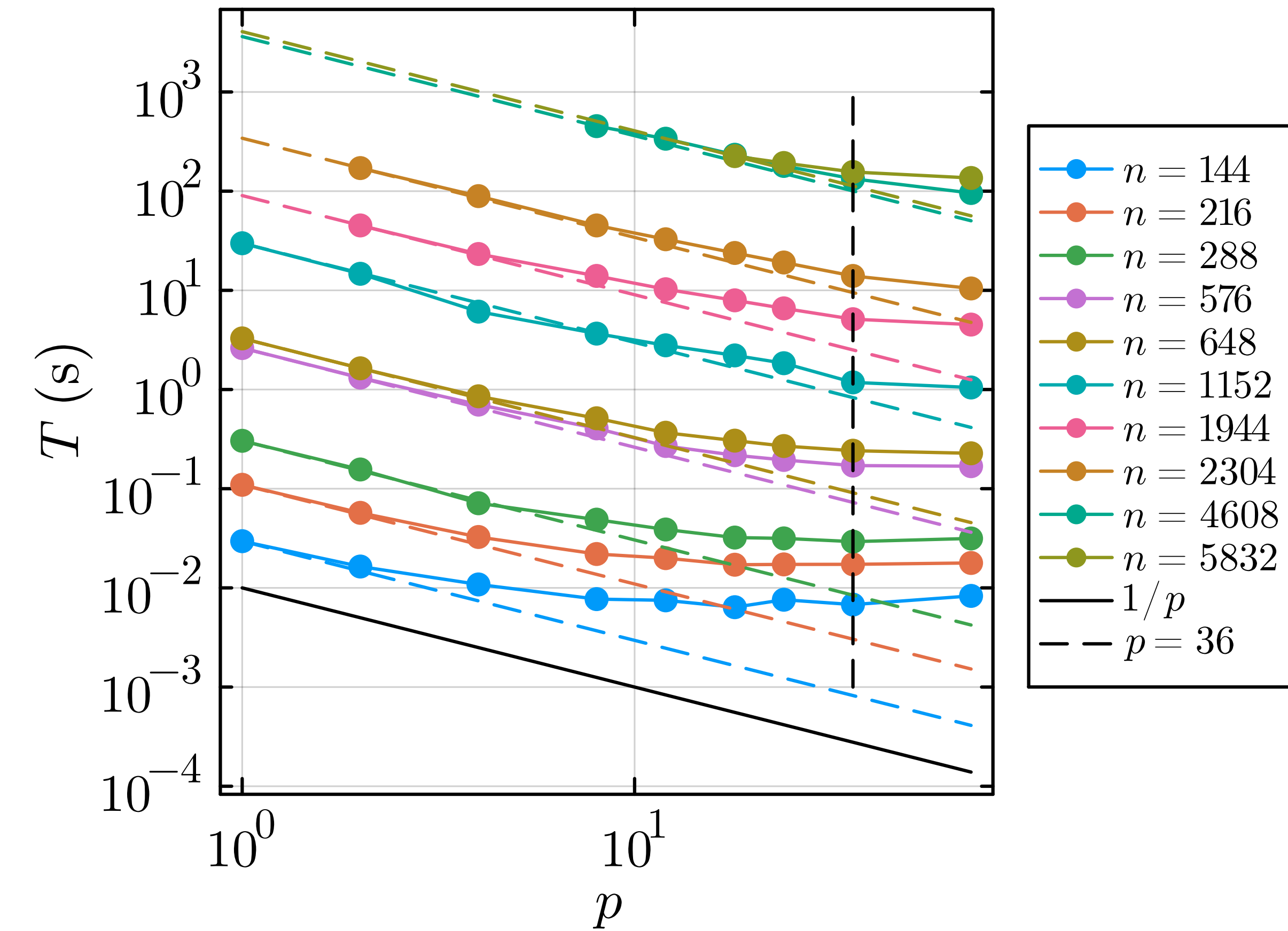
Verification of code

Initial matrix A , eigenvalues Λ and eigenvectors Z stored in HDF5 file, then imported into Julia program. Very good agreement with the LAPACK results (eigvals function).

```
1 using LinearAlgebra
2 using HDF5
3 using Glob
4 vn=[144 216 288 576 648 1152 1944 2304 4608 5832]
5 for n in vn
6     @show n
7     mat_files=glob("initial_matrix_mpi_n=$(n)*", "project_eigen/data")
8     eigenvector_files=glob("eigenvectors_mpi_n=$(n)*", "project_eigen/data")
9     eigenvalue_files=glob("eigenvalues_mpi_n=$(n)*", "project_eigen/data")
10    f=h5open(mat_files[1], "r")
11    A=reshape(read(f, "vector"), n, n)
12    f=h5open(eigenvector_files[1], "r")
13    Z=reshape(read(f, "vector"), n, n)
14    f=h5open(eigenvalue_files[1], "r")
15     $\Lambda$ =read(f, "vector")
16    @show maximum(abs.(sort( $\Lambda$ ) .- eigvals(A)) )
17    @show maximum(abs.(sort(diag(Z * A * Z')) .- eigvals(A)) )
18    @show maximum(abs.(Z * A * Z' - diagm(diag(Z * A * Z'))))
19 end
```

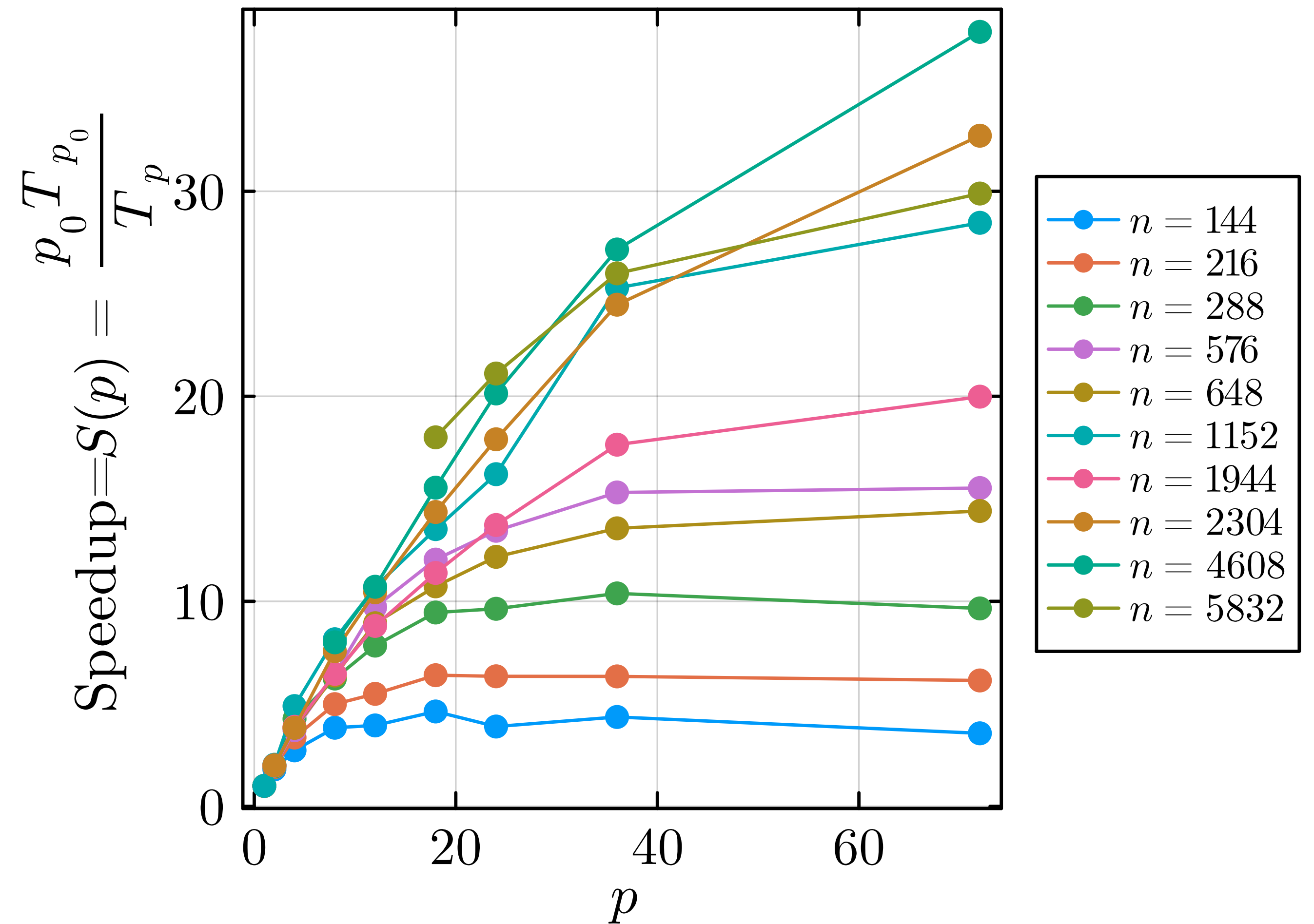
```
n = 288
maximum(abs.(sort( $\Lambda$ ) .- eigvals(A))) = 1.332986743207698e-8
maximum(abs.(sort(diag(Z * A * Z')) .- eigvals(A))) = 1.3329873649325918e-8
maximum(abs.(Z * A * Z' - diagm(diag(Z * A * Z')))) = 9.9986826042342e-6
n = 576
maximum(abs.(sort( $\Lambda$ ) .- eigvals(A))) = 1.4312097285085201e-8
maximum(abs.(sort(diag(Z * A * Z')) .- eigvals(A))) = 1.4312062646126833e-8
maximum(abs.(Z * A * Z' - diagm(diag(Z * A * Z')))) = 9.999606401078776e-6
n = 648
maximum(abs.(sort( $\Lambda$ ) .- eigvals(A))) = 2.310219215928555e-8
maximum(abs.(sort(diag(Z * A * Z')) .- eigvals(A))) = 2.3102003865460574e-8
maximum(abs.(Z * A * Z' - diagm(diag(Z * A * Z')))) = 9.999795908592624e-6
n = 1152
maximum(abs.(sort( $\Lambda$ ) .- eigvals(A))) = 2.0072217665756398e-8
maximum(abs.(sort(diag(Z * A * Z')) .- eigvals(A))) = 2.0038548598222405e-8
maximum(abs.(Z * A * Z' - diagm(diag(Z * A * Z')))) = 9.999966434588725e-6
n = 1944
maximum(abs.(sort( $\Lambda$ ) .- eigvals(A))) = 8.35581133173946e-8
maximum(abs.(sort(diag(Z * A * Z')) .- eigvals(A))) = 8.355868885701057e-8
maximum(abs.(Z * A * Z' - diagm(diag(Z * A * Z')))) = 9.999950384503853e-6
n = 2304
maximum(abs.(sort( $\Lambda$ ) .- eigvals(A))) = 1.2094758639058512e-7
maximum(abs.(sort(diag(Z * A * Z')) .- eigvals(A))) = 1.209475899432988e-7
maximum(abs.(Z * A * Z' - diagm(diag(Z * A * Z')))) = 9.999997767601299e-6
n = 4608
maximum(abs.(sort( $\Lambda$ ) .- eigvals(A))) = 6.744308578277014e-8
maximum(abs.(sort(diag(Z * A * Z')) .- eigvals(A))) = 6.74432234504252e-8
maximum(abs.(Z * A * Z' - diagm(diag(Z * A * Z')))) = 9.99999447541533e-6
n = 5832
maximum(abs.(sort( $\Lambda$ ) .- eigvals(A))) = 1.35119833544195e-7
maximum(abs.(sort(diag(Z * A * Z')) .- eigvals(A))) = 1.3512007512872515e-7
maximum(abs.(Z * A * Z' - diagm(diag(Z * A * Z')))) = 9.99999528942372e-6
```

Parallel runtime



Strong scaling

- Calculation of speedup: for larger n , where it would take too long to run serial code, assume lowest number of processors (p_0) has perfect speedup ($S(p_0) = p_0$) in order to estimate speedup.

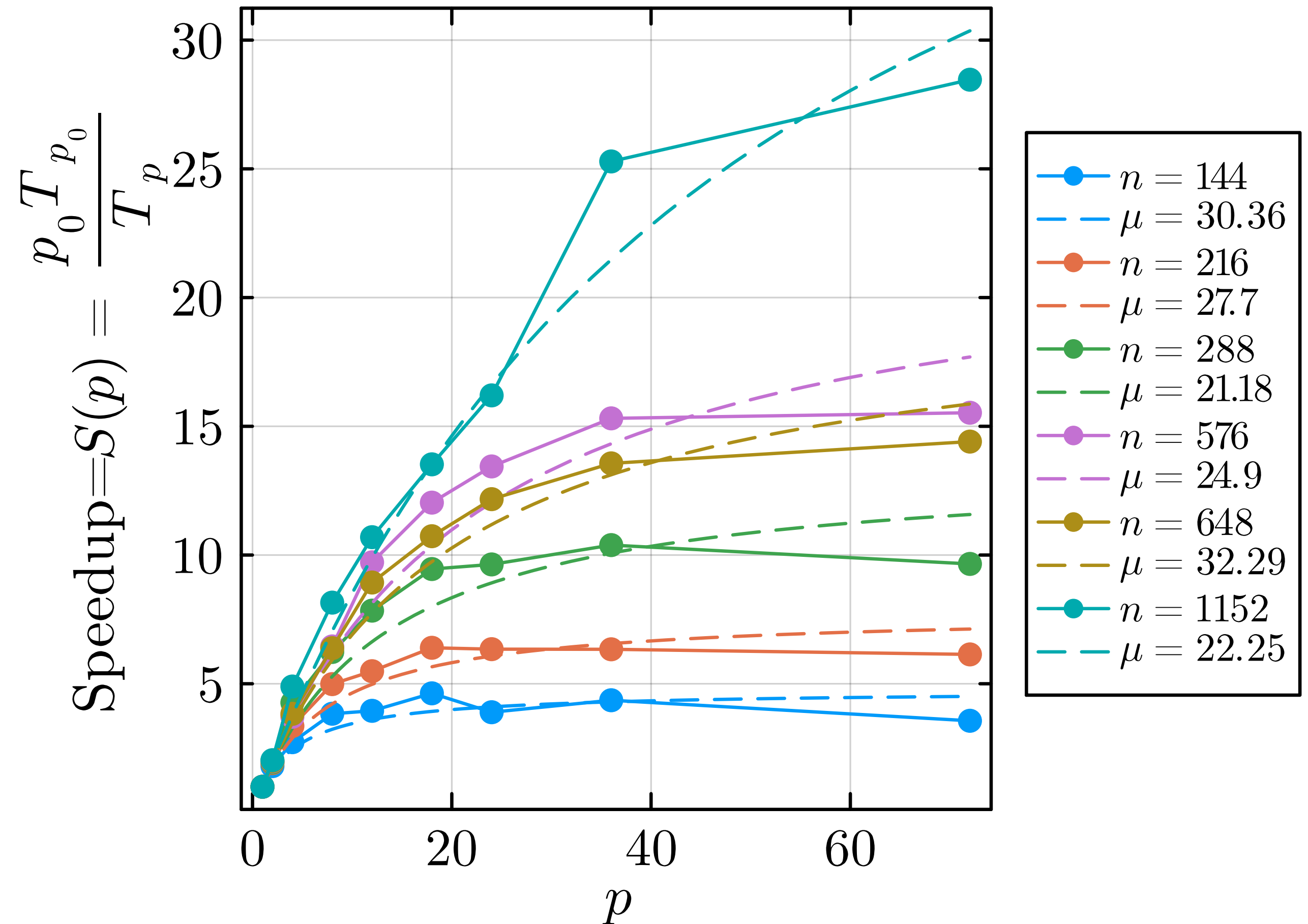


Strong scaling (Amdahl's law)

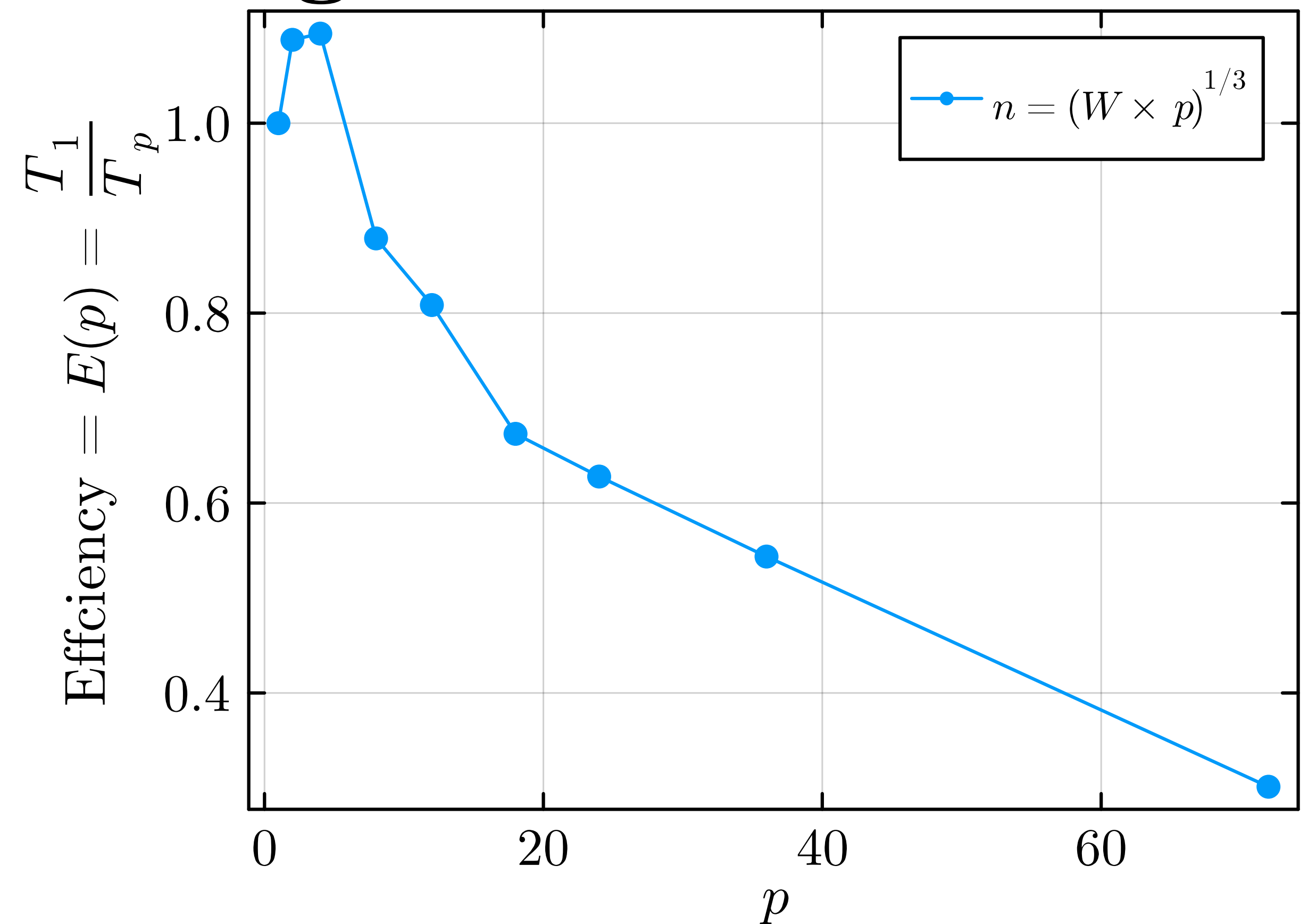
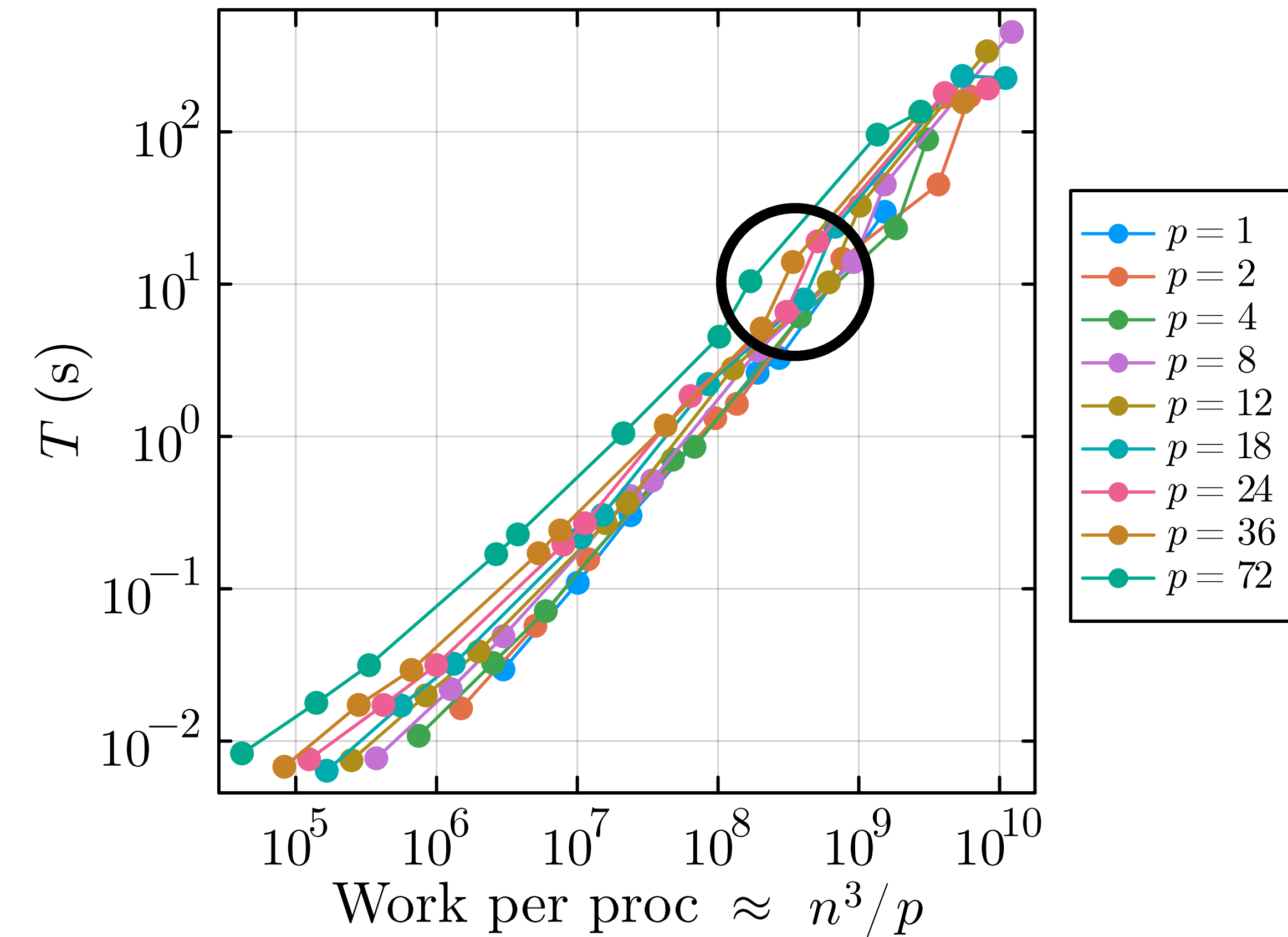
- Fit to Amdahl's law

$$S(p) = \frac{p}{1 + \frac{p\mu}{n} \left(1 - \frac{1}{p}\right)}$$

- Some qualitative agreement for $20 \leq \mu \leq 30$. More precise scaling analysis may require going to sub-leading orders in n .



Weak scaling



$$W = 5 \times 10^8 = \text{cte.}$$

Efficiency decreases during weak scaling.

Explanations: serial portion, and time cost of transferring data of size $O(n)$ between processors.